CSE-2206: Microcontroller Lab

I2C Communication with BMP280

Bare-Metal Implementation on STM32F446RE

A Comprehensive Function-by-Function Guide

Assignment 03 — LAB 02(A)

- **MCU:** STM32F446RE (Nucleo-64)
- **Sensor:** BMP280 (Bosch)
- **Protocol:** I2C (Standard Mode, 100 kHz)
- **Approach:** Bare-Metal (No HAL / No Library)

Lab Exam Preparation Guide

Contents

1. What Is I2C? — The Big Picture

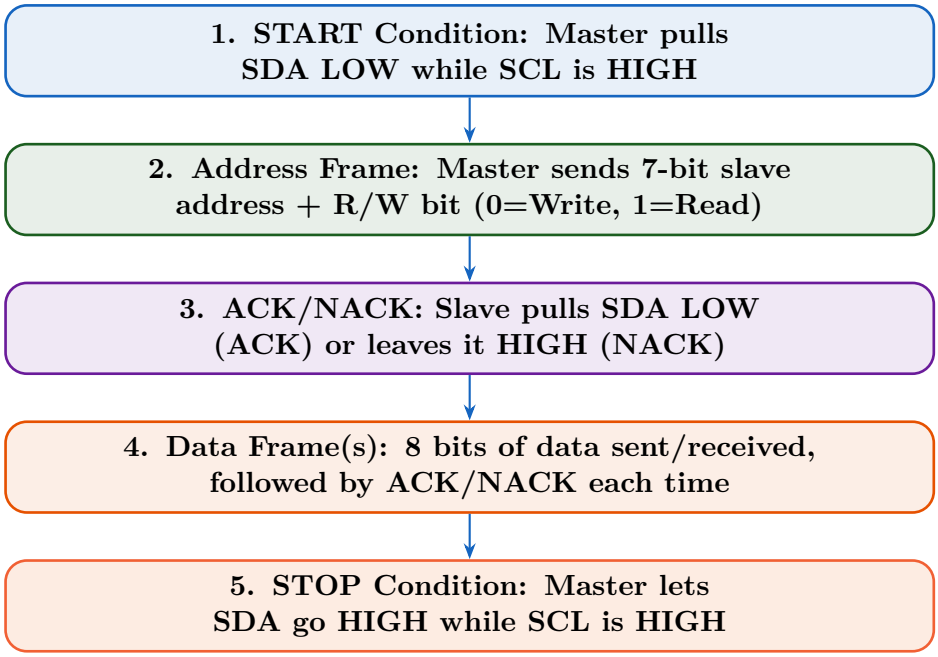
Understanding I2C in Simple Terms

I2C (Inter-Integrated Circuit) is a **two-wire** communication protocol that lets a **master** device (our STM32 microcontroller) talk to one or more **slave** devices (our BMP280 sensor) using just two lines:

- **SCL (Serial Clock Line)** — The master generates a clock signal here. Every "tick" of this clock means one bit of data is being transferred.
- **SDA (Serial Data Line)** — Actual data flows here, in both directions (master ↔ slave).

Think of it like a **telephone call**: the master dials a number (slave address), waits for the other party to pick up (ACK), then either speaks (writes data) or listens (reads data).

1.1 How I2C Works — Step by Step



1.2 Why I2C? (Compared to Other Protocols)

Feature	I2C	SPI	UART
Wires Needed	**2** (SDA, SCL)	4+ (MOSI, MISO, SCK, CS)	2 (TX, RX)
Multi-Slave?	**Yes** (via address)	Yes (extra CS per slave)	No (point-to-point)
Speed	Up to 3.4 Mbps	Up to 50+ Mbps	Up to 5 Mbps
Complexity	Medium	Low	Low
Acknowledgment	**Built-in** (ACK/NACK)	None	None

Why BMP280 uses I2C

The BMP280 sensor only needs to send a few bytes of data (temperature, pressure). I2C's low wire count (just 2 wires!) makes wiring simple, and its built-in addressing lets you connect multiple sensors on the same bus.

2. Hardware Wiring & Pin Configuration

Physical Connections

BMP280 Pin	Nucleo Pin	Purpose
VCC	3V3	Power supply (3.3V)
GND	GND	Ground reference
SCL	PB8 / D15	I2C clock line
SDA	PB9 / D14	I2C data line
CSB	Wired to VCC	Selects I2C mode (HIGH = I2C, LOW = SPI)
SDO	Wired to GND	Sets I2C address to `0x76` (LOW = 0x76, HIGH = 0x77)

Critical Wiring Note

- **CSB must be HIGH** (tied to VCC) to enable I2C mode. If CSB is LOW, the BMP280 switches to SPI mode.
- **SDO = GND** forces the slave address to `0x76`. If SDO = VCC, the address becomes `0x77`. This lets you connect **two** BMP280 sensors on the same I2C bus with different addresses!
- I2C lines need **open-drain outputs with pull-up resistors** so the bus stays HIGH when idle.

3. BMP280 Register Map & Macros

Every sensor has internal **registers** — small memory locations that you read from or write to in order to control the sensor or get data. Here are the registers used in our code:

Address	Macro Name	What It Does
0xD0	REGISTER_CHIP_ID	Returns chip ID: `0x58` for BMP280, `0x60` for BME280
0xE0	REGISTER_RESET	Write `0xB6` here to soft-reset the sensor
0xF2	REGISTER_CTRL_HUM	Humidity oversampling control (BME280 only)
0xF3	REGISTER_STATUS	Reports if a measurement is in progress
0xF4	REGISTER_CONTROL_MEASURE	Temperature/Pressure oversampling + operating mode
0xF5	REGISTER_CONFIGURE	IIR filter coefficient + standby time
0xF7	REGISTER_PRESSURE_MSB	Start of raw pressure data (3 bytes: 0xF7–0xF9)
0x88	REGISTER_CALIBRATIC	Start of calibration data block (24 bytes)

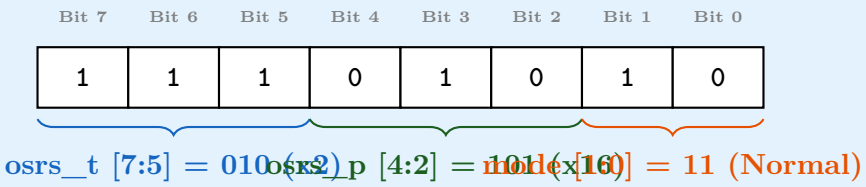
Register Macros in Code

```
#define BME_P280_REGISTER_CHIP_ID           0xD0
#define BME_P280_REGISTER_RESET             0xE0
#define BME_P280_REGISTER_CONTROL_MEASURE   0xF4
#define BME_P280_REGISTER_CONFIGURE         0xF5
#define BME_P280_REGISTER_PRESSURE_MSB      0xF7
#define BME_P280_REGISTER_CALIBRATION_START  0x88
```

```
#define BME_P280_RESET_VALUE                    0xB6

// Control Measure: Temp osrs x2, Press osrs x16, Normal mode
#define BME_P280_CONTROL_MEASURE_VALUE  ((2U << 5) | (5U << 2) | 3U)
// = 0b01_010_11 = 0x57

// Configure: IIR filter coeff x16, Standby 125ms
#define BME_P280_CONFIGURE_VALUE  ((4U << 5) | (4U << 2))
// = 0b100_100_00 = 0x90
```

Understanding the Control Measure Register (0xF4)

The `ctrl_meas` register at `0xF4` is packed as:



- **osrs_t = 010**: Temperature oversampling ×2 (16-bit resolution)
- **osrs_p = 101**: Pressure oversampling ×16 (20-bit ultra-high resolution)
- **mode = 11**: Normal mode (continuous measurement cycle)

4. Function: `PLL_CONFIG()` — System Clock Setup

4.1 Why Is This Function Needed?

When the STM32F446RE starts up, it runs on its **internal RC oscillator (HSI)** at only **16 MHz**. This function switches the system to the **PLL (Phase-Locked Loop)** to achieve **180 MHz** — the maximum clock speed. A faster clock means:

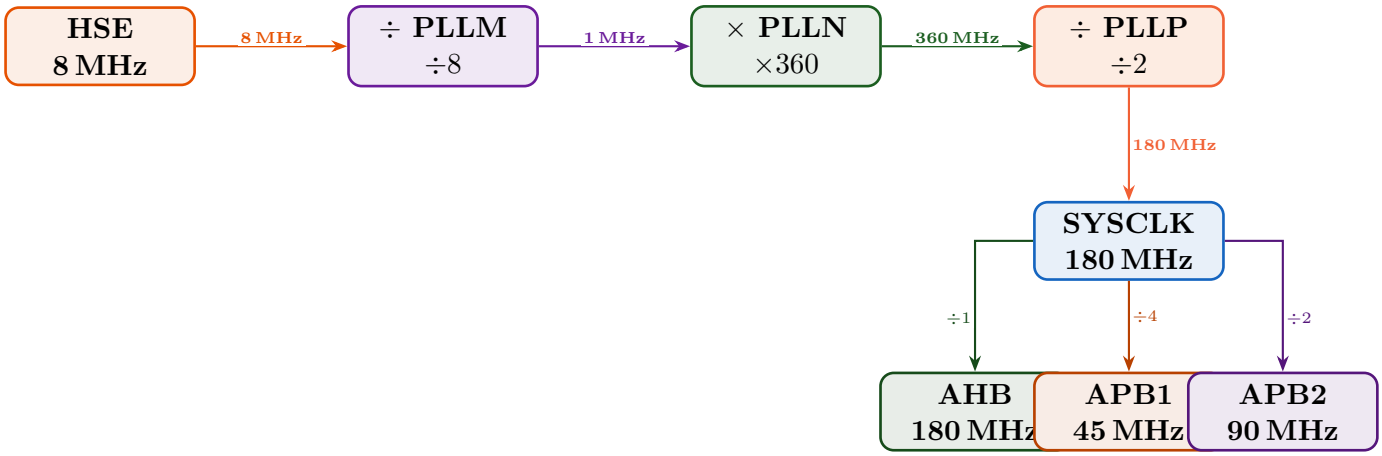
- Faster code execution
- Precise timing for I2C communication
- Accurate baud rate for USART debugging

4.2 Code Breakdown

PLL_CONFIG() — Full Code

```
void PLL_CONFIG(void)
{
    // Step 1: Configure Flash memory for high-speed access
    FLASH->ACR =
        FLASH_ACR_ICEN |        // Instruction Cache Enable
        FLASH_ACR_DCEN |        // Data Cache Enable
        FLASH_ACR_PRFTEN |      // Pre-fetch Enable
        FLASH_ACR_LATENCY_5WS; // 5 Wait States for 180MHz

    // Step 2: Enable the external 8MHz crystal (HSE)
    RCC->CR |= RCC_CR_HSEON;
    while (!(RCC->CR & RCC_CR_HSERDY)); // Wait until ready

    // Step 3: Configure PLL multipliers/dividers
    RCC->PLLCFGR =
        (8 << RCC_PLLCFGR_PLLM_Pos) |   // 8MHz / 8 = 1MHz
        (360 << RCC_PLLCFGR_PLLN_Pos) |  // 1MHz x 360 = 360MHz
           VCO
        (0 << RCC_PLLCFGR_PLLP_Pos) |    // 360MHz / 2 = 180MHz
        RCC_PLLCFGR_PLLSRC_HSE;          // Source = HSE

    // Step 4: Turn on PLL and wait
    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY));

    // Step 5: Configure bus pre-scalers
    RCC->CFGR |=
        RCC_CFGR_HPRE_DIV1 |  // AHB  = 180MHz
        RCC_CFGR_PPRE1_DIV4 | // APB1 = 45MHz (max)
        RCC_CFGR_PPRE2_DIV2;  // APB2 = 90MHz

    // Step 6: Switch system clock to PLL
    RCC->CFGR |= RCC_CFGR_SW_PLL;
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL);
}
```

4.3 Clock Tree Diagram



Key Takeaway

The I2C1 peripheral sits on the **APB1 bus at 45 MHz**. This value is critical because we use it directly when configuring I2C timing (`CR2 = 45`).

Alternative Implementation

Instead of using the PLL, you could:

1. **Use HSI directly (16 MHz)**: Simpler code, but I2C timing calculations change (`CR2 = 16`), and USART baud rate divisors change.
2. **Use the STM32 HAL library**: Call `HAL_RCC_OscConfig()` and `HAL_RCC_ClockConfig()` — the HAL calculates everything for you, but you lose understanding of what happens underneath.

5. Function: `GPIO_CONFIG()` — Pin Setup

5.1 Why Is This Function Needed?

Every pin on the STM32 is **multipurpose**. By default, they are just generic input pins. We must **explicitly configure** each pin for its role:

- **PA2** → USART2 TX (for serial debug output)
- **PB8** → I2C1 SCL (clock line)
- **PB9** → I2C1 SDA (data line)

5.2 Understanding GPIO Registers

For each pin, we configure **five** things:

Register	What It Controls	Our Setting for I2C Pins
MODER	Pin mode (Input/Output/AF/Analog)	`0b10` = Alternate Function
AFR	Which alternate function (0–15)	AF4 = I2C function
OTYPER	Output type (Push-Pull / Open-Drain)	Open-Drain (required for I2C!)
OSPEEDR	Output speed	Medium speed
PUPDR	Pull-up / Pull-down	Pull-up (keeps line HIGH when idle)

5.3 I2C Pin Configuration Code (PB8 — SCL)

Configuring PB8 as I2C1 SCL

```
// Step 1: Enable clock for GPIOB
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
__NOP(); __NOP(); // Brief delay for clock to stabilize

// Step 2: Set PB8 to Alternate Function mode (0b10)
GPIOB->MODER &= ~(3UL << 8*2);  // Clear bits [17:16]
GPIOB->MODER |=  (2UL << 8*2);  // Set to AF mode

// Step 3: Select AF4 (I2C1) in the Alternate Function Register
//   PB8 is in AFR[1] (high register, pins 8-15)
//   Position = (8-8)*4 = 0, so bits [3:0]
GPIOB->AFR[1] &= ~(0xF << (8-8)*4);  // Clear
GPIOB->AFR[1] |=  (4UL << (8-8)*4);  // AF4 = I2C

// Step 4: Set Open-Drain output (REQUIRED for I2C!)
GPIOB->OTYPER |= (1U << 8);  // 1 = Open-Drain

// Step 5: Set Medium speed
GPIOB->OSPEEDR |= (1U << (8*2));  // 0b01 = Medium

// Step 6: Enable Pull-Up (keeps SCL HIGH when idle)
GPIOB->PUPDR &= ~(3UL << 8*2);  // Clear
GPIOB->PUPDR |=  (1U << (8*2)); // 0b01 = Pull-Up
```

Why Open-Drain is Critical for I2C

I2C uses a **wired-AND** bus design:

- **Open-Drain** means a pin can only *pull LOW* or *float* (disconnect). It cannot actively

drive HIGH.

- The **pull-up resistor** pulls the line HIGH when nobody is pulling it LOW.
- This prevents electrical conflict when both master and slave try to control the same line.
- If we used **Push-Pull** instead, the master driving HIGH and slave driving LOW at the same time would cause a **short circuit**!

Alternative Implementation

- **HAL approach**: `HAL_GPIO_Init(GPIOB, &GPIO_InitStruct)` with `.Mode = GPIO_MODE_AF_OD`, `.Alternate = GPIO_AF4_I2C1`.
- **Different pins**: I2C1 can alternatively use **PB6** (SCL) and **PB7** (SDA) instead of PB8/PB9. You would change the MODER/AFR bit positions accordingly.
- **External pull-ups**: Instead of using the MCU's internal pull-ups (~40kΩ), you could add external 4.7kΩ resistors for more reliable communication, especially at higher speeds or with longer wires.

6. Function: `USART2_CONFIG()` & `USART2_SendString()`

6.1 Why Is This Needed?

USART2 is used for **debug output**. It sends text to your computer via the Nucleo's built-in USB-to-Serial converter. This is how we see the temperature and pressure readings on a serial terminal (like PuTTY, Tera Term, or the STM32CubeIDE console).

6.2 Baud Rate Calculation

Baud Rate Math

The baud rate register (`BRR`) value is computed as:

$$\text{USARTDIV} = \frac{f_{\text{APB1}}}{16 \times \text{BaudRate}} = \frac{45{,}000{,}000}{16 \times 115200} \approx 24.414$$

This is split into:

- **Mantissa** = 24 (integer part)
- **Fraction** = $0.414 \times 16 \approx 7$ (fractional part, scaled to 4 bits)

`BRR = (24 << 4) | 7 = 0x187` $\Rightarrow$ Actual baud rate $\approx$ 115,108 (close enough to 115,200).

USART Configuration & Send Function

```
void USART2_CONFIG(void)
{
    RCC->APB1ENR |= RCC_APB1ENR_USART2EN; // Enable clock
    __NOP();
    USART2->BRR = (24 << 4) | 7;          // 115200 baud
    USART2->CR1 = USART_CR1_TE | USART_CR1_UE; // TX + Enable
}

void USART2_SendString(const char *s)
{
    while (*s) {
        // Wait until transmit buffer is empty
        while (!(USART2->SR & USART_SR_TXE));
        USART2->DR = (uint8_t)(*s++); // Send one character
    }
    // Wait until transmission is complete
    while (!(USART2->SR & USART_SR_TC));
}
```

7. Functions: TIM6_CONFIG(), delay_us(), delay_ms()

7.1 Why Are These Needed?

The BMP280 sensor requires **precise timing delays** during initialization (e.g., waiting after a soft reset). The code uses **TIM6**, a basic hardware timer, to create microsecond-accurate delays.

Timer Setup & Delay Functions

```
void TIM6_CONFIG(void)
{
    RCC->APB1ENR |= RCC_APB1ENR_TIM6EN; // Enable TIM6 clock
    TIM6->PSC = 89U;        // 90MHz / (89+1) = 1MHz -> 1 tick = 1
       us
    TIM6->ARR = 0xFFFFU;    // Count up to 65535 (free-running)
    TIM6->EGR = TIM_EGR_UG; // Load PSC/ARR immediately
    TIM6->CR1 |= TIM_CR1_CEN; // Start counting
}

void delay_us(uint16_t us)
{
    TIM6->CNT = 0U;                    // Reset counter to 0
    while ((uint16_t)TIM6->CNT < us); // Wait until count reaches
       'us'
}
```

```
15
16 void delay_ms(uint32_t ms)
17 {
18     for (uint32_t i = 0; i < ms; i++)
19         delay_us(1000U);  // 1000 microseconds = 1 millisecond
20 }
```

Timer Math

TIM6 is on APB1 (45 MHz), but since the APB1 prescaler is ÷4 (not ÷1), the timer clock is actually $45 \times 2 = 90$ **MHz** (timers on APB1 get ×2 when the APB prescaler > 1). With `PSC = 89`: frequency = $90\,\text{MHz}/(89+1) = 1\,\text{MHz}$, so each tick = $1\,\mu s$.

Alternative Delay Approaches

1. **SysTick timer**: ARM Cortex-M4 has a built-in SysTick timer — simpler for delays but less flexible.
2. **HAL_Delay()**: The HAL library provides `HAL_Delay(ms)` using SysTick interrupt.
3. **Software loop delay**: Simple `for` loop counting cycles. Unreliable because compiler optimizations can change the timing.
4. **DWT cycle counter**: The Debug Watchpoint and Trace unit has a cycle counter (`DWT->CYCCNT`) for nanosecond-precision delays.

8. Function: I2C_CONFIG() — The Heart of I2C Setup

8.1 Why Is This Function Needed?

This function initializes the STM32's I2C1 peripheral hardware. Without this, the I2C hardware block is dormant and cannot generate clock signals or send/receive data.

I2C_CONFIG() — Full Code with Detailed Comments

```
1  void I2C_CONFIG(void)
2  {
3      // 1. Enable I2C1 peripheral clock
4      RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;
5
6      // 2. Reset the I2C peripheral (ensures clean state)
7      RCC->APB1RSTR |= RCC_APB1RSTR_I2C1RST;    // Assert reset
8      RCC->APB1RSTR &= ~RCC_APB1RSTR_I2C1RST;   // Release reset
9
10     // 3. Set peripheral input clock frequency (APB1 = 45MHz)
11     I2C1->CR2 = 45U;
```

```
12
13      // 4. Set Clock Control Register for 100kHz (Standard Mode)
14      //    CCR = APB1_freq / (2 * I2C_freq)
15      //        = 45,000,000 / (2 * 100,000) = 225
16      I2C1->CCR = 225U;
17
18      // 5. Set maximum rise time
19      //    TRISE = (APB1_freq_MHz * max_rise_time_us) + 1
20      //          = (45 * 1) + 1 = 46
21      I2C1->TRISE = 46U;
22
23      // 6. Enable the I2C peripheral
24      I2C1->CR1 |= I2C_CR1_PE;
25  }
```

8.2 Understanding Each Register

I2C Register Details

Register	Value	Explanation
CR2	45	Tells the I2C hardware "my input clock is 45 MHz." It needs this to calculate timing.
CCR	225	Clock Control Register. For standard mode (100 kHz), each half-period of SCL = 225/45 MHz = 5 μs. Full period = 10 μs = 100 kHz.
TRISE	46	Maximum allowed rise time. For standard mode, the I2C spec says $\leq$ 1000 ns. Formula: $(45 \times 1) + 1 = 46$.
CR1	PE bit	Peripheral Enable — turns on the I2C hardware.

CCR Calculation Deep Dive

For **Standard Mode (Sm)** at 100 kHz:

$$\text{CCR} = \frac{f_{\text{APB1}}}{2 \times f_{\text{I2C}}} = \frac{45{,}000{,}000}{2 \times 100{,}000} = 225$$

For **Fast Mode (Fm)** at 400 kHz with 2:1 duty cycle:

$$\text{CCR} = \frac{f_{\text{APB1}}}{3 \times f_{\text{I2C}}} = \frac{45{,}000{,}000}{3 \times 400{,}000} = 37.5 \approx 38$$

(You would also set the `F/S` bit and `DUTY` bit in CCR for Fast Mode.)

⇄ Alternative I2C Configuration Approaches

- **Fast Mode (400 kHz)**: Set `CCR |= I2C_CCR_FS` and recalculate CCR. Faster but needs better pull-ups.
- **HAL Library**: `HAL_I2C_Init(&hi2c1)` with `.ClockSpeed = 100000` handles all math for you.
- **I2C2 or I2C3**: The STM32F446 has three I2C peripherals. You could use I2C2 (PB10/PB11) or I2C3 (PA8/PC9) with the same logic, just changing the peripheral base address and pins.
- **DMA-based I2C**: For large data transfers, you could configure DMA to handle I2C reads/writes without CPU involvement.

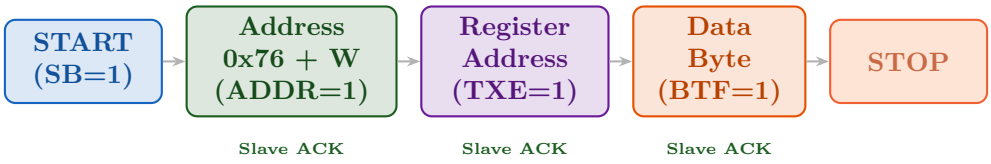
9. Function: `I2C_WRITE_BYTE()` — Writing to the Sensor

9.1 Why Is This Function Needed?

To control the BMP280, we need to **write values into its registers**. For example:

- Write `0xB6` to register `0xE0` to reset the sensor
- Write `0x57` to register `0xF4` to start measurements

9.2 I2C Write Transaction Diagram



</> I2C_WRITE_BYTE() — Annotated

```
1  void I2C_WRITE_BYTE(uint8_t reg, uint8_t data)
2  {
3      // Phase 1: Generate START condition on the bus
4      I2C1->CR1 |= I2C_CR1_START;
5      while (!(I2C1->SR1 & I2C_SR1_SB)); // Wait for SB (Start Bit)
           flag
6
7      // Phase 2: Send slave address (0x76) with WRITE bit (LSB = 0)
8      //   Address on wire = 0x76 << 1 | 0 = 0xEC
9      I2C1->DR = (BME_P280_I2C_ADDRESS << 1) | 0;
10     while (!(I2C1->SR1 & I2C_SR1_ADDR)); // Wait for ADDR flag (
           slave ACK'd)
11     (void)I2C1->SR2; // MUST read SR2 to clear ADDR flag (hardware
```

```
        requirement)
12
13      // Phase 3: Send the register address (e.g., 0xF4 for
            ctrl_meas)
14      I2C1->DR = reg;
15      while (!(I2C1->SR1 & I2C_SR1_TXE)); // Wait for TX buffer
            empty
16
17      // Phase 4: Send the data byte (e.g., 0x57)
18      I2C1->DR = data;
19      while (!(I2C1->SR1 & I2C_SR1_BTF)); // Wait for Byte Transfer
            Finished
20
21      // Phase 5: Generate STOP condition
22      I2C1->CR1 |= I2C_CR1_STOP;
23  }
```

Why Read SR2 to Clear ADDR?

This is an STM32 hardware quirk! The ADDR flag is cleared by a **specific sequence**: read SR1 (done by the `while` loop) then read SR2. The `(void)I2C1->SR2;` cast tells the compiler "I'm reading this on purpose, don't optimize it away." If you forget this step, the I2C will hang!

Alternative Write Approaches

- **HAL Library**: `HAL_I2C_Mem_Write(&hi2c1, 0x76<<1, reg, I2C_MEMADD_SIZE_8BIT, &data, 1, 100);`
- **Interrupt-driven**: Instead of polling flags in `while` loops, you could use I2C event interrupts (`I2C1_EV_IRQHandler`) for non-blocking writes.
- **DMA-driven**: For writing many bytes, configure DMA to feed bytes automatically without CPU intervention.

10. Function: `I2C_READ_BYTE()` — Reading a Single Byte

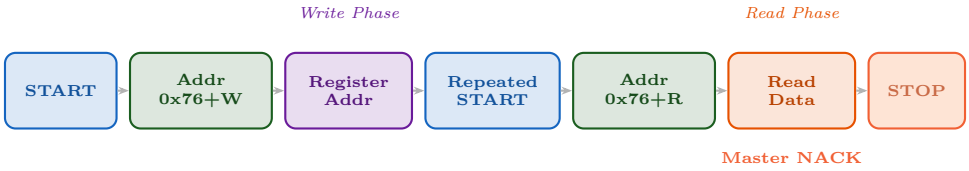
10.1 Why Is This Needed?

Reading from the BMP280 is a **two-phase process**:

1. **Write phase**: Tell the sensor *which register* you want to read (e.g., chip ID at `0xD0`)
2. **Read phase**: Switch direction and read the data back

This requires a **Repeated START** — instead of STOP then START, you send another START without releasing the bus.

10.2 I2C Read Transaction Diagram



I2C_READ_BYTE() — Annotated

```c
uint8_t I2C_READ_BYTE(uint8_t reg)
{
    uint8_t val = 0xFF;

    // === WRITE PHASE: Tell sensor which register to read ===
    I2C1->CR1 |= I2C_CR1_START;                    // START
    while (!(I2C1->SR1 & I2C_SR1_SB));

    I2C1->DR = (BME_P280_I2C_ADDRESS << 1) | 0;    // Address + WRITE
    while (!(I2C1->SR1 & I2C_SR1_ADDR));
    (void)I2C1->SR2;

    I2C1->DR = reg;                                 // Register address
    while (!(I2C1->SR1 & I2C_SR1_BTF));

    // === READ PHASE: Repeated START + read data ===
    I2C1->CR1 |= I2C_CR1_START;                    // Repeated START
    while (!(I2C1->SR1 & I2C_SR1_SB));

    I2C1->CR1 &= ~I2C_CR1_ACK;  // NACK (only reading 1 byte)
    I2C1->DR = (BME_P280_I2C_ADDRESS << 1) | 1;    // Address + READ
    while (!(I2C1->SR1 & I2C_SR1_ADDR));
    (void)I2C1->SR2;

    I2C1->CR1 |= I2C_CR1_STOP;                     // Queue STOP
    while (!(I2C1->SR1 & I2C_SR1_RXNE));           // Wait for data

    val = (uint8_t)I2C1->DR;                       // Read the byte
    return val;
}
```

Why NACK Before the Last Byte?

In I2C protocol:

- **ACK** (Acknowledge) means "I received the byte, *send me more*."
- **NACK** (Not Acknowledge) means "I received the byte, *stop sending*."

When reading only 1 byte, the master must send **NACK immediately** to tell the slave "don't send any more data." If you forget to clear the ACK flag, the slave will keep sending bytes, causing data corruption!

11. Function: `I2C_READ_BYTES()` — Multi-Byte Burst Read

11.1 Why Is This Needed?

The BMP280 calibration data spans **24 consecutive registers** (0x88–0x9F), and raw sensor data spans **8 registers** (0xF7–0xFE). Reading them one by one would be extremely slow. **Burst read** reads all of them in a single I2C transaction.

How Burst Read Works

The BMP280 has an **auto-increment** feature: once you send the starting register address, the sensor automatically advances its internal pointer to the next register after each byte. So you just keep reading!

I2C_READ_BYTES() — Multi-byte Read

```
void I2C_READ_BYTES(uint8_t start_reg, uint8_t *buf, uint8_t len)
{
    if (len == 0) return;

    // WRITE PHASE: Send the starting register address
    I2C1->CR1 |= I2C_CR1_START;
    while (!(I2C1->SR1 & I2C_SR1_SB));

    I2C1->DR = (BME_P280_I2C_ADDRESS << 1) | 0;  // Address + WRITE
    while (!(I2C1->SR1 & I2C_SR1_ADDR));
    (void)I2C1->SR2;

    I2C1->DR = start_reg;                        // Starting register
    while (!(I2C1->SR1 & I2C_SR1_BTF));

    // READ PHASE: Repeated START + read multiple bytes
```

```
    I2C1->CR1 |= I2C_CR1_START;
    while (!(I2C1->SR1 & I2C_SR1_SB));

    I2C1->CR1 |= I2C_CR1_ACK;  // Enable ACK (we want multiple
        bytes)
    I2C1->DR = (BME_P280_I2C_ADDRESS << 1) | 1;   // Address +
        READ
    while (!(I2C1->SR1 & I2C_SR1_ADDR));
    (void)I2C1->SR2;

    // Read loop
    for (uint8_t i = 0; i < len; i++)
    {
        if (i == (len - 1))
        {
            I2C1->CR1 &= ~I2C_CR1_ACK;   // NACK for last byte
            I2C1->CR1 |= I2C_CR1_STOP;    // Generate STOP
        }
        while (!(I2C1->SR1 & I2C_SR1_RXNE)); // Wait for data
        buf[i] = (uint8_t)I2C1->DR;           // Store byte
    }
}
```

ACK/NACK Strategy for Multi-Byte Reads

Byte 0	Byte 1	Byte 2	···	Byte N-2	Byte N-1
ACK	ACK	ACK		ACK	NACK + STOP

- Send **ACK** after every byte *except* the last one → tells slave "keep going"
- Send **NACK + STOP** after the last byte → tells slave "we're done"

12. Function: I2C_DEVICE_ALIVE() — Connection Check

12.1 Why Is This Needed?

Before trying to read data, we need to verify the BMP280 is **actually connected and responding**. This function sends just the address and checks if the slave responds with ACK.

I2C_DEVICE_ALIVE() — Connection Probe

```
int I2C_DEVICE_ALIVE(void)
{
    I2C1->CR1 |= I2C_CR1_START;           // START
```

```
    while (!(I2C1->SR1 & I2C_SR1_SB));

    I2C1->DR = (BME_P280_I2C_ADDRESS << 1) | 0;  // Address + WRITE

    uint32_t timeout = 200000;
    // Wait for either ADDR (ACK) or AF (NACK/no response)
    while (!(I2C1->SR1 & (I2C_SR1_ADDR | I2C_SR1_AF)) && --timeout);

    int alive = (I2C1->SR1 & I2C_SR1_ADDR) != 0;  // Check if ACK'd
    if (alive)
        (void)I2C1->SR2;           // Clear ADDR flag
    else
        I2C1->SR1 &= ~I2C_SR1_AF; // Clear AF (Acknowledge Failure) flag

    I2C1->CR1 |= I2C_CR1_STOP;
    delay_ms(2);
    return alive;  // 1 = alive, 0 = not responding
}
```

Understanding ADDR vs AF Flags

Flag	Full Name	Meaning
ADDR	Address Sent/Matched	The slave *responded* with an ACK → device is alive!
AF	Acknowledge Failure	No slave responded (NACK) → device not connected or wrong address.

Alternative Alive-Check Approaches

- **Read Chip ID**: Instead of just checking ACK, read the chip ID register (`0xD0`). If you get `0x58` (BMP280), the device is definitely alive and correct.
- **I2C Bus Scanner**: Loop through all 127 possible addresses and check which ones ACK. Useful for debugging when you don't know the sensor's address.
- **HAL**: `HAL_I2C_IsDeviceReady(&hi2c1, 0x76 << 1, 3, 100)` tries 3 times with 100 ms timeout.

13. Function: BME_P280_CALIBRATION_READ()

13.1 Why Are Calibration Values Needed?

Every BMP280 chip is **slightly different** due to manufacturing variations. Bosch stores **factory calibration coefficients** inside each sensor's non-volatile memory. These numbers are used in compensation formulas to convert raw ADC readings into accurate temperature and pressure values.

Calibration Structure

The calibration data is organized into three groups:

Group	Coefficients	Register Range	Data Type
Temperature	TEMP_1, 2, 3	0x88–0x8D	uint16, int16, int16
Pressure	PRESS_1 – 9	0x8E–0x9F	uint16 + 8×int16
Humidity	HUM_1 – 6	0xA1 + 0xE1–0xE7	Mixed types

Calibration Read (Temperature & Pressure)

```
void BME_P280_CALIBRATION_READ(void)
{
    uint8_t CALIB_VAL[24];
    // Read 24 consecutive bytes starting at 0x88
    I2C_READ_BYTES(BME_P280_REGISTER_CALIBRATION_START, CALIB_VAL,
        24);

    // Temperature calibration (bytes 0-5)
    //  Each parameter is 16-bit, stored LSB first (little-endian)
    CALIBRATION.TEMP_1 = (uint16_t)((CALIB_VAL[1] << 8) |
       CALIB_VAL[0]);
    CALIBRATION.TEMP_2 = (int16_t) ((CALIB_VAL[3] << 8) |
       CALIB_VAL[2]);
    CALIBRATION.TEMP_3 = (int16_t) ((CALIB_VAL[5] << 8) |
       CALIB_VAL[4]);

    // Pressure calibration (bytes 6-23)
    CALIBRATION.PRESS_1 = (uint16_t)((CALIB_VAL[7]  << 8) |
       CALIB_VAL[6]);
    CALIBRATION.PRESS_2 = (int16_t) ((CALIB_VAL[9]  << 8) |
       CALIB_VAL[8]);
    // ... PRESS_3 through PRESS_9 follow the same pattern ...
    CALIBRATION.PRESS_9 = (int16_t) ((CALIB_VAL[23] << 8) |
       CALIB_VAL[22]);
}
```

Little-Endian Byte Order

The BMP280 stores multi-byte values in **little-endian** format: the **least significant byte (LSB) comes first**. That's why we see:

```
value = (HIGH_BYTE « 8) | LOW_BYTE
```

For example, if register 0x88 = `0x34` and 0x89 = `0x12`, then TEMP_1 = `0x1234`.

14. Function: `BME_P280_INIT()` — Sensor Initialization

BME_P280_INIT() — Step-by-Step Initialization

```
void BME_P280_INIT(void)
{
    // Step 1: Soft-reset the sensor
    //   Writing 0xB6 to the reset register (0xE0)
    //   resets all internal registers to defaults
    I2C_WRITE_BYTE(BME_P280_REGISTER_RESET, BME_P280_RESET_VALUE);
    delay_ms(10); // Sensor needs ~2ms to reset, we give it 10ms

    // Step 2: Set the config register (0xF5)
    //   Value 0x90 = IIR filter x16, standby time 125ms
    I2C_WRITE_BYTE(BME_P280_REGISTER_CONFIGURE,
       BME_P280_CONFIGURE_VALUE);
    delay_ms(2);

    // Step 3: (BME280 only) Set humidity oversampling
    I2C_WRITE_BYTE(BME_P280_REGISTER_CTRL_HUM,
       BME_P280_CTRL_HUM_VALUE);
    delay_ms(2);

    // Step 4: Set measurement control (0xF4)
    //   Value 0x57 = Temp x2, Pressure x16, Normal mode
    //   IMPORTANT: ctrl_meas must be written AFTER ctrl_hum!
    I2C_WRITE_BYTE(BME_P280_REGISTER_CONTROL_MEASURE,
                   BME_P280_CONTROL_MEASURE_VALUE);
    delay_ms(2);
}
```

Register Write Order Matters!

According to the Bosch datasheet:

- `config` (0xF5) should be written **before** entering Normal mode
- `ctrl_hum` (0xF2) must be written **before** `ctrl_meas` (0xF4) — the humidity settings

only take effect when `ctrl_meas` is written

If you write them in the wrong order, the sensor may use old/default settings!

IIR Filter Explained (Config Register 0xF5)

The BMP280 has a built-in **IIR (Infinite Impulse Response) filter** that smooths out pressure readings by averaging with previous measurements:

$$\text{filtered} = \frac{(\text{coefficient} - 1) \times \text{previous} + \text{new}}{\text{coefficient}}$$

Our code sets coefficient = 16 (high filtering), which eliminates short-term fluctuations like door slams or wind gusts. For weather monitoring, this is ideal. For altitude tracking in a drone, you might want coefficient = 2 (faster response).

15. Compensation Functions — Converting Raw Data to Real Values

15.1 Why Compensation Is Needed

The BMP280's ADC gives **raw 20-bit numbers** that don't directly represent temperature or pressure. Bosch provides **fixed-point integer compensation formulas** (no floating-point!) that use the calibration coefficients to convert these raw values.

15.2 Temperature Compensation

BME_P280_BOSCH_TEMP_COMPENSATE()

```
int32_t BME_P280_BOSCH_TEMP_COMPENSATE(int32_t adc_T)
{
    int32_t var1, var2, T;

    // Linear correction term
    var1 = ((((adc_T >> 3) - ((int32_t)CALIBRATION.TEMP_1 << 1)))
              * ((int32_t)CALIBRATION.TEMP_2)) >> 11;

    // Quadratic correction term (handles non-linearity)
    var2 = (((((adc_T >> 4) - ((int32_t)CALIBRATION.TEMP_1))
              * ((adc_T >> 4) - ((int32_t)CALIBRATION.TEMP_1))) >>
                 12)
              * ((int32_t)CALIBRATION.TEMP_3)) >> 14;

    // t_fine: a global variable shared with pressure/humidity
       formulas
    t_fine = var1 + var2;
```

```
16
17      // Temperature in units of 0.01 degrees Celsius
18      T = (t_fine * 5 + 128) >> 8;
19
20      return T;  // e.g., 2534 means 25.34 degrees C
21  }
```

Understanding `t_fine`

The variable `t_fine` is the **most important intermediate value**. It represents a "fine temperature" that captures the sensor's current thermal state. Both the **pressure** and **humidity** compensation formulas use `t_fine` because:

- Pressure measurements are *temperature-dependent* (warm air is less dense)
- Humidity readings drift with temperature

Temperature must ALWAYS be compensated first, so that `t_fine` is available for the other formulas.

Fixed-Point Arithmetic

The formulas use **bit shifts** instead of division (`>> 8` instead of `/ 256`) because:

- STM32F4 has no floating-point division unit for 64-bit types
- Integer arithmetic is **deterministic** — no rounding errors
- Bit shifts execute in **1 clock cycle** vs. division which takes 2–12 cycles

The final result is in **hundredths of a degree**: 2534 = 25.34°C.

15.3 Pressure Compensation

BME_P280_BOSCH_PRESS_COMPENSATE() — Key Steps

```
1  uint32_t BME_P280_BOSCH_PRESS_COMPENSATE(int32_t adc_P)
2  {
3      int64_t var1, var2, p;
4
5      var1 = ((int64_t)t_fine) - 128000;  // Temperature correction
           base
6
7      // Higher-order polynomial corrections using PRESS_2 through
           PRESS_6
8      var2 = var1 * var1 * (int64_t)CALIBRATION.PRESS_6;
9      var2 = var2 + ((var1 * (int64_t)CALIBRATION.PRESS_5) << 17);
10     var2 = var2 + (((int64_t)CALIBRATION.PRESS_4) << 35);
11
12     var1 = ((var1 * var1 * (int64_t)CALIBRATION.PRESS_3) >> 8)
```

```
        + ((var1 * (int64_t)CALIBRATION.PRESS_2) << 12);
    var1 = (((((int64_t)1) << 47) + var1))
        * ((int64_t)CALIBRATION.PRESS_1) >> 33;

    if (var1 == 0) return 0;  // Guard: avoid division by zero

    p = 1048576 - adc_P;
    p = (((p << 31) - var2) * 3125) / var1;

    // Final corrections using PRESS_7, PRESS_8, PRESS_9
    var1 = (((int64_t)CALIBRATION.PRESS_9) * (p >> 13) * (p >> 13)
        ) >> 25;
    var2 = (((int64_t)CALIBRATION.PRESS_8) * p) >> 19;
    p = ((p + var1 + var2) >> 8) + (((int64_t)CALIBRATION.PRESS_7)
        << 4);

    return (uint32_t)(p >> 8);  // Pressure in Pascals
}
```

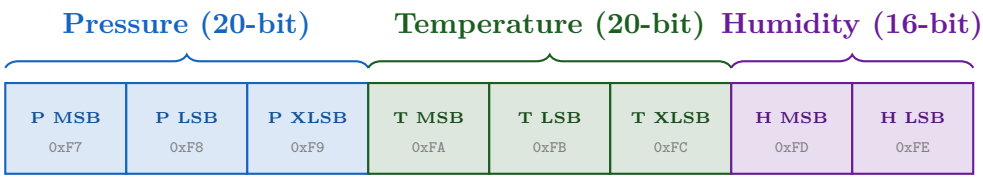
Pressure Units

- The function returns pressure in **Pascals (Pa)**
- Standard atmospheric pressure = 101325 Pa = 1013.25 hPa
- The code divides by 100 later to display in **hectopascals (hPa)**, which is the standard meteorological unit

16. Function: BME_P280_TEMP_PRESS_HUM()

16.1 How Raw Data Is Structured

Reading 8 bytes starting from `0xF7` gives us:



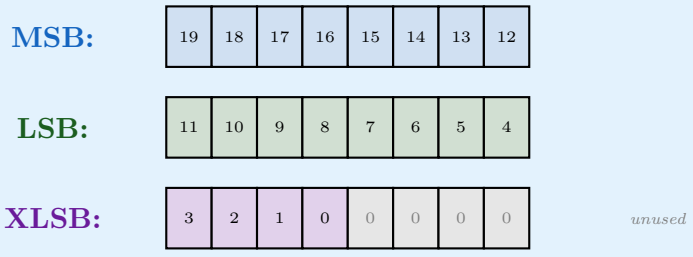
BME_P280_TEMP_PRESS_HUM() — Read & Display

```
void BME_P280_TEMP_PRESS_HUM(void)
{
    char MESSAGE[120];
    uint8_t VALUE[8];

    // Burst-read all 8 data registers at once
    I2C_READ_BYTES(BME_P280_REGISTER_PRESSURE_MSB, VALUE, 8);
```

```
8
9       // Reconstruct 20-bit raw pressure (MSB[19:12], LSB[11:4],
            XLSB[3:0])
10      int32_t P = ((int32_t)VALUE[0]  <<  12) |
11                  ((int32_t)VALUE[1]  <<  4)  |
12                  ((int32_t)VALUE[2]  >>  4);
13
14      // Reconstruct 20-bit raw temperature (same bit layout)
15      int32_t T = ((int32_t)VALUE[3]  <<  12) |
16                  ((int32_t)VALUE[4]  <<  4)  |
17                  ((int32_t)VALUE[5]  >>  4);
18
19      // Reconstruct 16-bit raw humidity
20      int32_t H = ((int32_t)VALUE[6]  <<  8) | VALUE[7];
21
22      // Apply Bosch compensation formulas
23      int32_t  TEMP     = BME_P280_BOSCH_TEMP_COMPENSATE(T);  //
            Must be first!
24      uint32_t PRESSURE = BME_P280_BOSCH_PRESS_COMPENSATE(P);
25      uint32_t HUMIDITY = BME_P280_BOSCH_HUM_COMPENSATE(H);
26
27      // Format for display: separate integer and fraction parts
28      int32_t  TEMP_FINAL = TEMP / 100;           // e.g., 25
29      int32_t  TEMP_FRAC  = (TEMP < 0 ? -TEMP : TEMP) % 100; // e.g
            ., 34
30      uint32_t PRESS_HPA  = PRESSURE / 100;       // e.g., 1013
31      uint32_t PRESS_FRAC = PRESSURE % 100;       // e.g., 25
32
33      sprintf(MESSAGE, "| %6ld.%02ld C | %6lu.%02lu hPa | ...\r\n",
            ...);
34      USART2_SendString(MESSAGE);
35  }
```

20-bit Raw Data Reconstruction

The BMP280 stores raw values across 3 bytes but only uses 20 bits:



Formula: `raw = (MSB « 12) | (LSB « 4) | (XLSB » 4)`

17. Function: SENSOR_CHECK() — Identification Routine

SENSOR_CHECK() — Verify and Identify the Sensor

```
void SENSOR_CHECK(void)
{
    // Step 1: Check if any device responds at address 0x76
    if (!I2C_DEVICE_ALIVE()) {
        USART2_SendString("[!] ACK not received. Going idle...\r\n
            ");
        while (1); // Halt forever -- no point continuing
    }

    // Step 2: Read the chip ID register (0xD0)
    uint8_t id = I2C_READ_BYTE(BME_P280_REGISTER_CHIP_ID);

    // Step 3: Identify which sensor it is
    if (id == CHIP_ADDRESS_BME280)       // 0x60
        USART2_SendString("[*] BME280 detected (has humidity)\r\n"
            );
    else if (id == CHIP_ADDRESS_BMP280)  // 0x58
        USART2_SendString("[*] BMP280 detected\r\n");
    else {
        // Unknown chip -- halt
        while (1);
    }
}
```

BME280 vs BMP280 — Key Differences

Feature	BMP280	BME280
Chip ID	0x58	0x60
Temperature	✓ Yes	✓ Yes
Pressure	✓ Yes	✓ Yes
Humidity	✗ No	✓ Yes
Price	Lower	Slightly higher

Note: The problem statement mentions BME280, but the lab provides **BMP280**. The

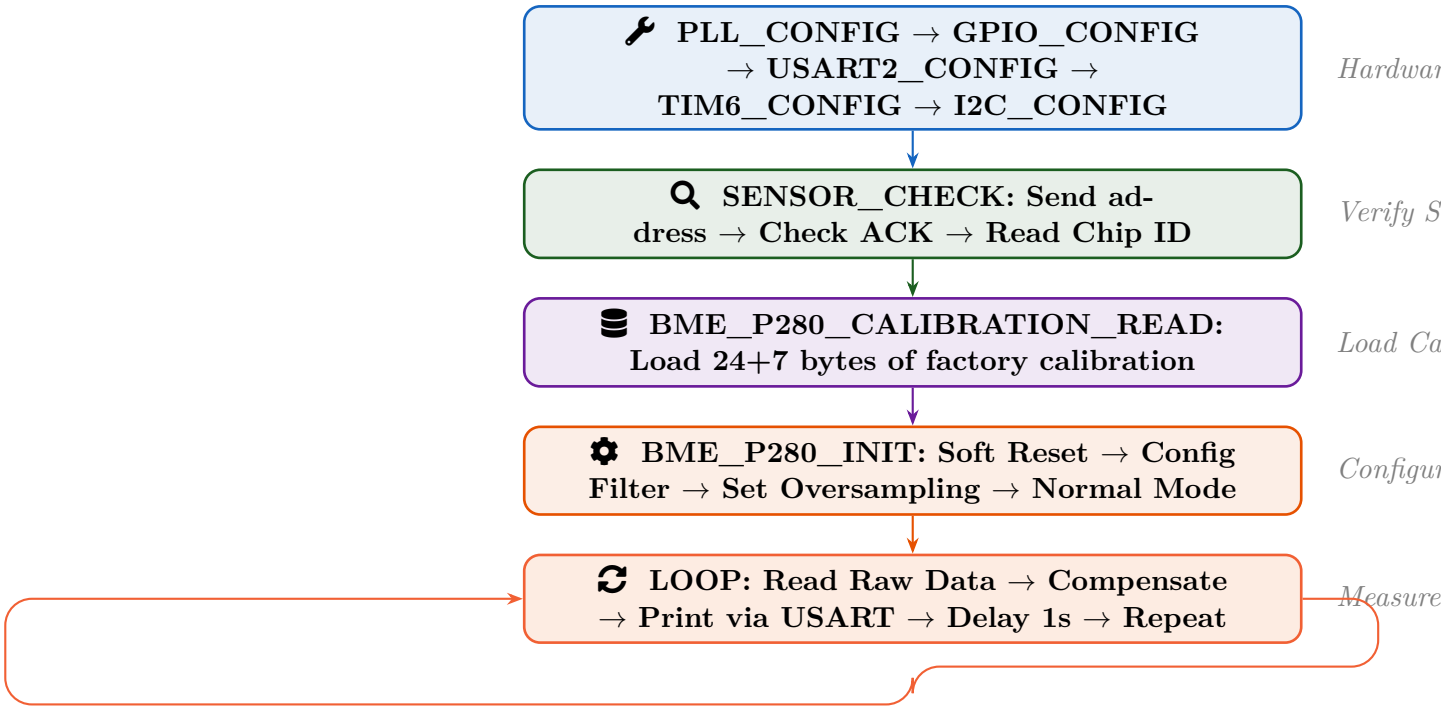
code handles both — it checks the chip ID and adapts. For BMP280, humidity readings will show as 0 since the sensor lacks that capability.

18. Function: `main()` — Putting It All Together

The Main Function — Execution Flow

```c
int main(void)
{
    // Phase 1: Hardware Initialization
    PLL_CONFIG();      // Set system clock to 180MHz
    GPIO_CONFIG();     // Configure I2C + USART pins
    USART2_CONFIG();   // Set up serial debug output
    TIM6_CONFIG();     // Set up microsecond delay timer
    I2C_CONFIG();      // Initialize I2C1 peripheral at 100kHz

    USART2_SendString("LAB 02: BME/P-280 Communication by I2C\r\n"
       );

    // Phase 2: Sensor Verification
    SENSOR_CHECK();    // Verify device is alive + identify chip

    // Phase 3: Load Calibration Data
    BME_P280_CALIBRATION_READ();  // Read factory calibration
       coefficients

    // Phase 4: Configure Sensor
    BME_P280_INIT();  // Reset, set oversampling, normal mode

    // Phase 5: Continuous Measurement Loop
    while (1) {
        BME_P280_TEMP_PRESS_HUM();  // Read + compensate + display
        delay_ms(1000);             // Wait 1 second
    }
}
```

18.1 Complete Execution Flow Diagram



19. I2C Communication Waveform Summary

Complete I2C Write Transaction: Writing 0x57 to Register 0xF4

S = START condition, **P** = STOP condition, **W** = Write bit (0), **ACK** = Acknowledge from slave.

20. Quick Reference — Function Summary Table

Function	Category	Purpose
`PLL_CONFIG()`	Clock	Sets system clock to 180 MHz via PLL using HSE
`GPIO_CONFIG()`	GPIO	Configures PA2 (USART TX), PB8 (SCL), PB9 (SDA)
`USART2_CONFIG()`	Debug	Initializes USART2 at 115200 baud for serial output
`USART2_SendString()`	Debug	Sends a null-terminated string character by character
`TIM6_CONFIG()`	Timer	Configures TIM6 for 1 μs resolution free-running
`delay_us()` / `delay_ms()`	Timer	Blocking microsecond / millisecond delay
`I2C_CONFIG()`	I2C	Initializes I2C1 at 100 kHz standard mode
`I2C_WRITE_BYTE()`	I2C	Writes one byte to a specified register
`I2C_READ_BYTE()`	I2C	Reads one byte from a specified register
`I2C_READ_BYTES()`	I2C	Burst-reads multiple bytes from consecutive registers
`I2C_DEVICE_ALIVE()`	I2C	Checks if slave responds with ACK
`SENSOR_CHECK()`	Sensor	Verifies and identifies BMP280 / BME280
`CALIBRATION_READ()`	Sensor	Loads factory calibration coefficients
`BME_P280_INIT()`	Sensor	Resets sensor, configures oversampling and mode
`TEMP_COMPENSATE()`	Math	Converts raw ADC to temperature (°C × 100)
`PRESS_COMPENSATE()`	Math	Converts raw ADC to pressure (Pa)
`HUM_COMPENSATE()`	Math	Converts raw ADC to humidity (%RH × 1024)
`TEMP_PRESS_HUM()`	Main	Reads, compensates, and prints all sensor data

21. Common Lab Exam Questions & Answers

Frequently Asked Questions

Q1: Why do I2C lines use open-drain with pull-ups?

Because I2C is a *wired-AND* bus. Multiple devices share the same wires. Open-drain prevents short circuits when two devices try to control the line simultaneously. Pull-ups keep the default state HIGH.

Q2: What is the difference between START and Repeated START?

A **START** begins a new transaction after the bus is idle. A **Repeated START** changes direction (write→read) *without releasing the bus*. This prevents another master from hijacking the bus between your write (register address) and read (data) phases.

Q3: Why must temperature be compensated before pressure?

Because the pressure formula depends on `t_fine`, a global variable set during temperature compensation. Pressure measurements are temperature-dependent — warm air exerts different pressure than cold air at the same altitude.

Q4: How is the I2C address determined to be 0x76?

The BMP280 has a configurable address pin (SDO). When SDO is connected to **GND**, the address is `0x76`. When connected to **VCC**, it becomes `0x77`. This allows two BMP280 sensors on the same I2C bus.

Q5: What does the IIR filter do in the BMP280?

The IIR (Infinite Impulse Response) filter smooths pressure readings by weighing previous measurements. Higher coefficients (like x16 in our code) produce smoother data but slower response. This is ideal for weather stations but not for fast-changing altitude measurements.

Q6: Why do we read SR2 after checking ADDR flag?

It's an STM32 hardware design requirement. The ADDR flag is cleared by the specific sequence of reading SR1 then SR2. Forgetting to read SR2 leaves ADDR set, and the I2C peripheral will hang waiting for the flag to be cleared.

Q7: What's the difference between BMP280 and BME280?

BMP280 measures **temperature and pressure** only. BME280 adds **humidity sensing**. They are pin-compatible and use the same I2C protocol, but have different chip IDs (0x58 vs 0x60).

Q8: Why is CCR set to 225?

CCR (Clock Control Register) determines SCL frequency. For 100 kHz standard mode with 45 MHz APB1 clock: CCR = 45,000,000 / (2 × 100,000) = 225. Each half-period of SCL = 225 × (1/45MHz) = 5 μs, so full period = 10 μs = 100 kHz.

Good Luck on Your Lab Exam! ★

This document covers every function in the bare-metal I2C BMP280 implementation.
Generated for CSE-2206 Microcontroller Lab — Assignment 03, LAB_02(A).